In [ ]:
```python
import os
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader, Dataset
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from typing import List
```

In [ ]:
```python
torch.manual_seed(0)
np.random.seed(0)
```

In [ ]:
```python
class ResNetEncd(nn.Module):
    def __init__(self, base="resnet18", proj_dim=128):
        super().__init__()
        backbone = getattr(torchvision.models, base)(pretrained=False)
        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        feat_dim = backbone.fc.in_features
        self.projection = nn.Sequential(
            nn.Linear(feat_dim, feat_dim),
            nn.ReLU(inplace=True),
            nn.Linear(feat_dim, proj_dim)
        )

    def forward(self, x, return_projection: bool = True):
        feat = self.backbone(x)
        feat = torch.flatten(feat, 1)
        if return_projection:
            proj = self.projection(feat)
            return feat, proj
        return feat

    def represent(self, x):
        return self.forward(x, return_projection=False)
```

In [ ]:
```python
def compute_typicality(features: np.ndarray, k: int = 20) -> np.ndarray:
    nbrs = NearestNeighbors(n_neighbors=min(k + 1, len(features)), algorithm='au
    distances, _ = nbrs.kneighbors(features)  # distances: [N, k+1]
    distances = distances[:, 1:]
    avg_sq_dist = (distances ** 2).mean(axis=1)
    typicality = 1.0 / (avg_sq_dist + 1e-12)
    return typicality

def typiclust_selection(
    features: np.ndarray,
    initial_labeled: List[int],
    budget_total: int,
    batch_size_per_round: int,
    k_typicality: int = 20,
    random_state: int = 0,
) -> List[int]:
    N = features.shape[0]
    rng = np.random.RandomState(random_state)

    labeled = set(initial_labeled)
```

```
    unlabeled = set(range(N)) - labeled

    typicality = compute_typicality(features, k=k_typicality)

    while len(labeled) < budget_total:
        current_L = list(labeled)
        current_U = list(unlabeled)

        n_clusters = len(current_L) + batch_size_per_round
        n_clusters = min(n_clusters, len(current_L) + len(current_U))  # safety

        kmeans = KMeans(n_clusters=n_clusters, random_state=rng.randint(1e9))
        cluster_labels = kmeans.fit_predict(features)

        cluster_has_label = np.zeros(n_clusters, dtype=bool)
        for idx in current_L:
            c = cluster_labels[idx]
            cluster_has_label[c] = True

        uncovered_clusters = [c for c in range(n_clusters) if not cluster_has_la

        cluster_sizes = {c: np.sum(cluster_labels == c) for c in uncovered_clust
        sorted_clusters = sorted(uncovered_clusters, key=lambda c: -cluster_size

        new_indices = []
        for c in sorted_clusters:
            if len(labeled) + len(new_indices) >= budget_total:
                break
            cluster_idx = np.where(cluster_labels == c)[0]
            cluster_idx = [i for i in cluster_idx if i in unlabeled]
            if not cluster_idx:
                continue
            cluster_typ = typicality[cluster_idx]
            best_local = cluster_idx[int(np.argmax(cluster_typ))]
            new_indices.append(best_local)

        for idx in new_indices:
            labeled.add(idx)
            unlabeled.remove(idx)

        if not new_indices:
            break

    return sorted(labeled)
```

In [ ]:

```
def train_supervised(
    encoder: ResNetEncd,
    dataset: Dataset,
    device: torch.device,
    batch_size: int = 256,
    epochs: int = 500,
    lr: float = 1e-3,
    num_classes: int = 10,
    resume_path: str = None
) -> ResNetEncd:
    # Train the encoder using traditional supervision on labels
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
```

```
        num_workers=4,
        drop_last=True
    )

    classifier = nn.Linear(128, num_classes).to(device)
    encoder = encoder.to(device)

    loss_fn = nn.CrossEntropyLoss().to(device)
    optimizer = torch.optim.Adam(list(encoder.parameters()) + list(classifier.pa

    start_epoch = 0
    if resume_path is not None and os.path.exists(resume_path):
        print(f"Resuming Supervised training from checkpoint: {resume_path}")
        checkpoint = torch.load(resume_path, map_location=device)
        encoder.load_state_dict(checkpoint["encoder"])
        classifier.load_state_dict(checkpoint["classifier"])
        optimizer.load_state_dict(checkpoint["optimizer"])
        start_epoch = checkpoint["epoch"] + 1
        print(f"Resumed from epoch {start_epoch}")

    encoder.train()
    classifier.train()

    sup_losses = []

    for epoch in range(start_epoch, epochs):
        running_loss = 0.0
        for x, y in loader:
            x = x.to(device)
            y = y.to(device)

            _, z = encoder(x, return_projection=True)
            preds = classifier(z)

            loss = loss_fn(preds, y)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * x.size(0)

        epoch_loss = running_loss / len(loader.dataset)
        print(f"Epoch {epoch+1}/{epochs} - Supervised LOSS : {epoch_loss:.4f}")
        sup_losses.append(epoch_loss)

        if (epoch+1) % 40 == 0:
            os.makedirs("supervised_budget_results/sup_checkpoints", exist_ok=Tr
            ckpt_path = f"supervised_budget_results/sup_checkpoints/sup_epoch_{e
            torch.save({
                "encoder": encoder.state_dict(),
                "classifier": classifier.state_dict(),
                "optimizer": optimizer.state_dict(),
                "epoch": epoch
            }, ckpt_path)
            print(f"Checkpoint Saved at {epoch+1} -> {ckpt_path}")

    os.makedirs("supervised_budget_results", exist_ok=True)
    np.save("supervised_budget_results/sup_loss.npy", np.array(sup_losses))
    return encoder
```

In [ ]:

```python
def extract_features(
    encoder: ResNetEncd,
    dataset: torchvision.datasets.CIFAR10,
    device: torch.device,
    batch_size: int = 256
) -> np.ndarray:
    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=4
    )

    encoder.eval()
    all_features = []

    with torch.no_grad():
        for x, _ in loader:
            x = x.to(device)
            features = encoder.represent(x)
            all_features.append(features.cpu().numpy())

    features = np.concatenate(all_features, axis=0)
    return features


def run_pipeline_supervised(
    data_root: str = "./data",
    budget_total: int = 100,
    batch_size_per_round: int = 10,
    sup_epochs: int = 500,
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    sup_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_supervised(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256,
        epochs=sup_epochs,
        lr=1e-3,
    )
```

```
    features = extract_features(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256
    )

    labeled_indices = typiclust_selection(
        features=features,
        initial_labeled=[],
        budget_total=budget_total,
        batch_size_per_round=batch_size_per_round,
        k_typicality=20,
        random_state=0,
    )

    os.makedirs("supervised_budget_results", exist_ok=True)
    np.save("supervised_budget_results/typiclust_supervised.npy", np.array(label
    print(f"Selected {len(labeled_indices)} labeled points (supervised)")

    return labeled_indices

def generate_and_save_typiclust_selections(
        budgets=[10, 20, 40, 80],
        batch_size_per_round=10,
        sup_epochs=500,
        data_root="./data"
):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    base_transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(
            (0.4914, 0.4822, 0.4465),
            (0.2470, 0.2435, 0.2616)
        ),
    ])

    sup_dataset = torchvision.datasets.CIFAR10(
        root=data_root,
        train=True,
        download=True,
        transform=base_transform
    )

    encoder = ResNetEncd(base="resnet18", proj_dim=128)
    encoder = train_supervised(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256,
        epochs=sup_epochs,
        lr=1e-3,
    )

    features = extract_features(
        encoder=encoder,
        dataset=sup_dataset,
        device=device,
        batch_size=256
```

```
    )

    os.makedirs("supervised_budget_results", exist_ok=True)

    for B in budgets:
        labeled_indices = typiclust_selection(
            features=features,
            initial_labeled=[],
            budget_total=B,
            batch_size_per_round=batch_size_per_round,
            k_typicality=20,
            random_state=0,
        )

        np.save(f"supervised_budget_results/typiclust_B{B}.npy", np.array(labele
        print(f"Saved TypiClust selection for B={B} → supervised_budget_results/
```

In [ ]:

```
if __name__ == '__main__':
    # run_pipeline_supervised(sup_epochs=500)
    generate_and_save_typiclust_selections(sup_epochs=500)
```